Howard University
College of Engineering & Architecture
Department of Electrical Engineering & Computer Science

EECE 306-01: Fundamentals of Electromagnetics Laboratory

Dr. Su Yan, su.yan@howard.edu

Lab 7: Writing Your Own Poisson Solver

Objectives

By the end of this session you will have

- derived and implemented the five point finite difference stencil,
- compared three iterative solvers and found the best relaxation parameter,
- verified a solver on a problem with no naturally occurring exact answer,
- measured a second order convergence rate and learned what a first order result would have meant.

1. A different way of getting the field

Everything so far has summed contributions from known sources. That approach cannot handle the problem that actually matters in engineering, where the charge on a conductor is *not* known in advance and instead the potential on it is fixed.

For that you solve the differential equation directly,

$$\nabla \cdot (\varepsilon \nabla V) = -\rho_v$$

subject to boundary conditions, which in the uniform case reduces to Poisson's equation

$$\nabla^2 V = -\frac{\rho_v}{\varepsilon}$$

A graphical tool exists that will do this for you by clicking. This course does not use it. Clicking through a solver teaches you where its buttons are. Deriving the stencil and watching your own iteration converge teaches you what the solver is, why it sometimes fails, and how to tell a converged answer from a plausible one. Those are the things you will still need in ten years.

2. Part I. The five point stencil

Start from the second central difference in one dimension,

$$\left.\frac{\partial^2 V}{\partial x^2}\right|_{i,j} \approx \frac{V_{i+1,j} - 2V_{i,j} + V_{i-1,j}}{h_x^2}$$

Adding the y counterpart gives the discrete Laplacian, and setting it equal to $-\rho_{i,j}/\varepsilon$ gives, after rearrangement for unequal spacings,

$$V_{i,j} = \frac{\dfrac{V_{i+1,j} + V_{i-1,j}}{h_x^2} + \dfrac{V_{i,j+1} + V_{i,j-1}}{h_y^2} + \dfrac{\rho_{i,j}}{\varepsilon}}{\dfrac{2}{h_x^2} + \dfrac{2}{h_y^2}}$$

Derive this yourself before implementing it. The special case $h_x = h_y$ collapses to the familiar statement that the potential at a node is the average of its four neighbours plus a source term, and that averaging property is worth remembering because it explains why Laplace solutions have no interior extrema.

Implement `em.op.laplacian(f, r, h)` as well, the pointwise operator, which gives you an independent way to check a solved field satisfies the equation it was supposed to satisfy.

3. Part II. The grid and the solver

`G = em.solve.grid2d(xlim, ylim, Nx, Ny)` returns a struct with `G.x`, `G.y`, `G.X`, `G.Y`, `G.hx` and `G.hy`.

`[V, info] = em.solve.poisson2d(G, rho, epsr, mask, Vfix, opts)` solves the problem.

- `rho` is an array, a scalar, or a handle `@(X,Y)`.
- `epsr` is an array or a scalar. It stays scalar today and becomes piecewise next week.
- `mask` is a logical array, true where the potential is fixed.
- `Vfix` supplies the values, read only where `mask` is true.
- `opts.method` is one of `'jacobi'`, `'gs'`, `'sor'`, `'direct'`, with `opts.tol`, `opts.maxit` and `opts.omega`.

Outer boundary nodes not covered by `mask` take the homogeneous Neumann condition $\partial V/\partial n = 0$.

`info.resHist` is the residual at every iteration, and `info.iter` and `info.time` record cost.

Scaffolding

A skeleton of `em.solve.poisson2d` is provided on Canvas. The iteration loop, the convergence check, the boundary handling and the `info` bookkeeping are written for you. What is left blank is the stencil and the update rule, which is the part that teaches something. Nobody is expected to write a solver from an empty file in 110 minutes.

4. Part III. Three iterations and a direct solve

Jacobi updates every node from the previous sweep. Simple, and slow.

Gauss-Seidel uses new values as soon as they are available within a sweep. Typically about twice as fast, and it cannot be vectorized in the same naive way, which is itself instructive.

SOR over-relaxes, $V^{\text{new}} = V^{\text{old}} + \omega(V^{\text{GS}} - V^{\text{old}})$, with $1 < \omega < 2$. For a good ω it is dramatically faster than either.

Required figure, the three residual histories on one semilog plot for the same problem and the same tolerance.

Required study, scan ω from 1.0 to 1.99 and plot iteration count against ω. The curve has a sharp minimum. Report your optimum, then repeat on a grid twice as fine and report it again. The optimal ω depends on the grid, and noticing that is the point of doing it twice.

Direct assembles the sparse matrix and calls backslash. It is in the API for two reasons. It gives you an answer you can trust while debugging the iterative methods, and it lets you find out for yourself where direct solution stops being affordable. Time both against grid size, plot the two curves, and report the crossover on your machine.

5. Part IV. The method of manufactured solutions

This is the most important technique in the course, and today is where it is introduced properly.

The difficulty with verifying a solver is that problems with exact solutions are rare, simple, and unrepresentative. Testing only against them leaves the general case unchecked.

The manufactured solutions method inverts the problem. Rather than searching for a problem whose answer is known, *choose the answer first* and then work out what source term would have produced it.

Take, on the unit square with $V = 0$ on all four edges,

$$V_{\text{exact}}(x, y) = \sin(\pi x)\sin(\pi y)$$

Apply the Laplacian,

$$\nabla^2 V_{\text{exact}} = -2\pi^2 \sin(\pi x)\sin(\pi y)$$

and compare with Poisson's equation $\nabla^2 V = -\rho/\varepsilon$ to read off

$$\rho(x, y) = 2\pi^2 \varepsilon \sin(\pi x)\sin(\pi y)$$

Feed that ρ into your solver with homogeneous Dirichlet conditions on the boundary. The computed V must reproduce $\sin(\pi x)\sin(\pi y)$ to discretization accuracy. Implement this as `em.test.mms2d(Vexact, epsr)`, returning the required source term.

Why professionals use this

Every other verification technique in this course has a precondition. A closed form check needs a closed form. A symmetry check needs a symmetry. A conservation check needs a conserved quantity in a form you can integrate.
The manufactured solution has none. You can choose any smooth function you like, on any domain, with any permittivity distribution, and derive the source that produces it. That means the method works on a problem shaped exactly like the one you actually care about, rather than on a simplified stand-in that might not exercise the same code paths.
This is standard practice in professional solver development and in code verification standards used across computational engineering. It is not an academic exercise.
One caution. A manufactured solution verifies that your code solves the equation you programmed. It says nothing about whether that equation describes the physics you meant. Verification and validation are different questions, and this technique answers only the first.

5.1 The convergence study

Solve on a sequence of grids, measure the maximum absolute error against V_{exact}, and pass the result to `em.test.convergence` from Lab 4. The observed order must come out near 2, matching the second order stencil.

If you measure order one

An observed order near 1 instead of 2 is the single most common failure in this lab, and it is almost always a boundary condition problem. The usual causes are applying the Dirichlet value at the wrong node index, treating a boundary node as an interior node in the residual, or a Neumann edge implemented with a one sided first order difference where a second order treatment was required.
Knowing this symptom will save you an hour. More generally, a convergence rate that comes in below the theoretical rate is a bug signal, not a curiosity, and treating it as such is the habit

worth taking away from today.

6. Part V. A check that closes the loop

Take your solved potential, wrap it in an interpolant to make a function handle, compute $\vec{\boldsymbol{E}} = -\nabla V$ with `em.op.grad`, and then compute the flux of $\vec{\boldsymbol{D}}$ through a closed contour inside the domain using `em.field.flux` from Lab 5.

Gauss's law must still hold, and now it is being applied to a field that came from a solver rather than from a source sum. That is exactly why `em.field.flux` was specified to take a function handle back in Lab 5.

Required tests

Write `tests/test_lab07.m` containing at least these checks.

1. The manufactured solution is reproduced with maximum error below 10^{-3} on a 101×101 grid.
2. Observed convergence order between 1.8 and 2.2 on a grid refinement sequence.
3. Jacobi, Gauss-Seidel, SOR and direct all agree with each other to the solver tolerance on the same problem.
4. SOR at your reported optimal ω needs fewer iterations than Gauss-Seidel, which needs fewer than Jacobi.
5. A pure Laplace problem, no source, with $V = 1$ on one edge and $V = 0$ on the opposite edge and Neumann on the other two, gives a linear potential profile.
6. The solved potential has no interior maximum or minimum in a source free region.
7. Gauss's law holds for the solver output as described in Part V.

Deliverables

Submit through Canvas by the start of the next lab session.

1. **Toolbox snapshot**, `Team`*`NN`*`_Lab07.zip`.
2. **Lab notebook**, published script exported to PDF, three to five pages, with the three residual histories on one plot, the ω scan on two grids, the timing crossover between iterative and direct, and the convergence table.
3. **Test output** pasted into the notebook.

Looking ahead

Next week is the midterm milestone. The solver acquires dielectrics, the toolbox computes its first quantity that the textbook cannot give in closed form, and there is a formal report in place of the weekly notebook. Read Sections 2.11 through 2.13 of Wentworth before you come, and bring a working solver.